AP 2026 Exam Notes

Written exam · Mon 9 November 2026 · Eksamenshuset

Print these notes and bring them to the exam. All aids are allowed except generative AI and the internet. You also get `ap-notes.pdf` and `compendium.pdf` on the exam computer.

The notes are ordered like the exam: **§G General**, then **one section per assignment**, then **§W7 week 7**. Each section has three parts:

1. **Facts:** the semantics you need to reason with.
2. **Model answers** to every report question (★) and dummy-exam question (D) for that assignment. The exam reuses these ideas.
3. **Variants:** the same idea with a different expression, which is how the exam is most likely to ask.

Expression results marked ✓ were run in GHC (A1 on the group hand-in, week 7 on the compendium solution code).

§0	Using these notes in the exam	**2**
§G	General questions	**4**
§A1	Evaluator with `Either` (week 1, compendium A1 handout p.24–25)	**5**
§A2	Monadic evaluator (week 2, compendium p.35–37, A2 text p.40–46)	**7**
§A3	Parsing (week 3, A3 text p.74–80, week 7 parser p.220–223)	**9**
§A4	Free monads (week 4, A4 text p.109–120, Monad p.218–219 for the week 7 version)	**11**
§A5	Property-based testing (week 5, A5 text p.148–152)	**13**
§A6/W6	Concurrency, GenServer, SPC (compendium p.166–201, notes p.177–217)	**15**
§W7	Week 7: tuples, while, `&&`/`||`, three interpreters (p.202–239)	**17**
+	My additions (fill in by hand or here)	**19**

> **Printing:** A4, single-sided, scale 100% (“Actual size”), and staple top-left. Every section starts on a new page, so you can put a sticky tab on the first page of each one.

§0 Using these notes in the exam

Recipe for every question (≈ 5 min each)

1. Which assignment or week? → jump to that section.
2. Find the matching model answer or variant. If none matches, use the **Facts** of that section.
3. Write **the answer + one reason**, in 1–3 sentences. The dummy solutions are this short.
4. Never leave a question blank. The right concept with a short reason earns credit even if a detail is wrong.

Answer templates

- *"What does X evaluate to?"* → "`X` evaluates to **V**, because **rule**." (e.g. "…because the loop bound is evaluated in the outer scope, where `i = -1`.")
- *"Which error?"* → name the error and **which subexpression is evaluated first**.
- *"Is it always this result?"* → pure and simulated interpreters are **deterministic**; the concurrent interpreter is **not** (it depends on thread scheduling).
- *"What changes if the code is defined like this?"* → name the **observable** difference: result value, which error, which effects survive, termination.
- *"Advantage/disadvantage of X vs Y"* → one concrete point each, plus what a user of the program could observe.
- *"Rewrite/sketch code"* → small code, then one sentence on what it now does.

Where to look things up

Compendium (`compendium.pdf`), page numbers:

Topic	Exercises	Exercise solutions	Assignment text (★ questions)	Assignment handout code
W1 / A1 evaluator	1–6	7–12	13–20 (★ p.18–19)	21–27 (Eval 24–25)
W2 / A2 monads	28–32	33–39 (Eval 35–37)	40–46 (★ p.45–46)	47–55
W3 / A3 parsing	56–65	66–73 (Parser 70–71)	74–80 (★ p.79–80)	81–88
W4 / A4 free monads	89–96	97–108 (Monad 105–106)	109–120 (★ p.119)	121–133
W5 / A5 QuickCheck	134–137	138–147	148–152 (★ p.151)	153–165
W6 / A6 concurrency	166–179	180–188 (GenServer 181, SPC 182–186)	189–194 (★ p.193–194)	195–201
W7 exam exercises	202–208	209–239	—	—

Week 7 solution files: AST 210 · Eval 211–212 · InterpConcurrent 213–214 · InterpPure 215 · **InterpSim 216** · InterpStep 217 · Monad 218–219 · Parser 220–223 · GenServer 224–225 · **KVDB 226** · SPC 227–230 · Tests 231–239.

Course notes (`ap-notes.pdf`), PDF page numbers:

Topic	Page	Topic	Page
Type classes, Functor	9, 12	Free monads	79
Monoids	15	Recipe for other effects	85
Monads, Applicative	23	Error monad	92
Monad laws	28	IO, exceptions	95–101
Reader monad	31	Free monads with IO	109

Topic	Page	Topic	Page
State monad, Reader+State	36, 38	Async programming with free monads	117
Parsing (parser monad)	40–46	Property-based testing	124
Whitespace	49	Generators, recursive, sized	136–141
Left recursion	58	Seeing what you test / coverage	149, 151
Operator precedence	62	Shrinking	154
Megaparsec	64	Testing stateful systems	157
Parsing comments	71	Concurrency, channels	177–179
Monads revisited	73	GenServer implementation	183
		Timeouts	197
		Concurrency with free monads	202

§G General questions

G1 (D 1.1) Effects as data (free monads) vs using `IO` directly.

- *Advantage:* the same program can be **interpreted in different ways**: pure (for testing, deterministic, no files), IO (real database/terminal), simulated concurrency. `eval` does not know how effects are implemented (separation of concerns).
- *Disadvantage:* more boilerplate (an effect type, a `Functor` instance, and a case in every interpreter for every effect). It is slower, because the effect tree is built and then walked. Every interpreter must handle every effect.

G2 (D 1.2) A property passes 100 tests. Why add `cover`? Passing says nothing by itself: the 100 inputs may all be similar and trivial (e.g. no variables, never a division by zero). `cover p cond label` with `checkCoverage` **makes the test fail if too few inputs hit the interesting case**, so it checks that the generator is strong enough.

G3 (D 1.3) Remove left recursion. `E ::= E "-" T | T` becomes

```
E  ::= T E'
E' ::= "-" T E' | ε
T  ::= "1" | "2" | "3"
```

Left associativity comes from passing the tree built so far as an **accumulator**:

```
pE = do x <- pT; pE' x
pE' x = choice [ do lString "-"; y <- pT; pE' (Sub x y)
               , pure x ]          -- pure x must be LAST (it always succeeds)
```

`1-2-3` → `pE' 1` → `pE' (Sub 1 2)` → `Sub (Sub 1 2) 3` = `(1-2)-3`. In EBNF shorthand: `E ::= T { "-" T }`, folded to the left.

G4 Lexical (static) vs dynamic scope. Lexical: a function body sees the variables of the place where the **function was defined** (the closure's env). Dynamic: it sees the variables of the place where it is **called**. `let y = 1 in let f = \x -> x + y in let y = 100 in f 1` → lexical **2** ✓, dynamic **101**. APL uses lexical scope for variables. A4's `break` is **dynamically** scoped (innermost loop at run time).

G5 Monad laws (course notes p.28):

- Left identity: `pure a >>= f = f a`
- Right identity: `m >>= pure = m`
- Associativity: `(m >>= f) >>= g = m >>= (\x -> f x >>= g)`

If an `EvalM` breaks one, refactorings that look harmless change behaviour. For example, `do x <- m; pure x` is no longer `m`, or regrouping `do` blocks into helper functions changes the result.

G6 Why `fmap = liftM` and `(<*>) = ap`? Once `>>=` and `pure` are written, Functor and Applicative come for free. They are then guaranteed to agree with the Monad instance and to satisfy their laws.

G7 Evaluation order matters only when something can fail, diverge or have effects. If both operands terminate successfully without effects, the order is unobservable (D 2.1.1.3).

G8 Eager APL. `Let`, `Apply` arguments and `put` values are evaluated **before** use, even if never used: `let x = 1/0 in 5` → `Division by zero` ✓. A lazy APL would give 5.

G9 Deterministic or not? Pure and simulated interpreters: deterministic (same answer every run). Concurrent interpreter / SPC: nondeterministic, the result can depend on scheduling. **Tests** for nondeterministic code must accept every allowed result, or use programs whose result cannot vary.

G10 Synchronous vs asynchronous call (GenServer). Sync = `requestReply`: send the message with a reply channel, then block until the server replies. Async = `sendTo`: send and return immediately, without a reply. Async is faster and cannot deadlock the caller, but the caller does not know when (or if) the message was handled.

§A1 Evaluator with `Either` (week 1, compendium A1 handout p.24–25)

Facts

- `eval :: Env -> Exp -> Either Error Val`. `Env = [(VName, Val)]` with the newest binding first.
- **Binary operators:** evaluate the left operand, then the right. The **first error wins**, and because of Haskell's laziness the right operand is not evaluated at all if the left fails. Type errors (`"Non-integer operand"`) are only detected after both operands succeed.
- `Div` by 0 → `"Division by zero"`. `Pow` with a negative exponent → `"Negative exponent"`. `Eql` on int vs bool → `"Invalid operands to equality"` ✓ (an error, not `false`).
- `If`: the condition must be a bool. Only the chosen branch is evaluated.
- `Let x e1 e2`: **eager**. `e1` is evaluated first, and its error propagates even if `x` is unused.
- `ForLoop (p, init) (i, bound) body`:
  1. evaluate `init`, then `bound`, both **in the outer environment**;
  2. the bound must be an integer, otherwise `"Non-integral loop bound"`;
  3. run the body `n` times with `i = 0..n-1`, each time with `p` bound to the previous result;
  4. return the final `p`. If `n ≤ 0`, the body is never evaluated. `p` and `i` are not visible after the loop ✓.
- `Lambda v body` → `ValFun env v body`. The closure captures the **definition** environment → lexical scope.
- `Apply e1 e2` (group hand-in): evaluate `e1`, then `e2`, then check that `e1` is a `ValFun`. Then evaluate the body in `closureEnv + [v ↦ arg]`. The argument is evaluated even if the body does not use it (call-by-value).
- `TryCatch e1 e2`: evaluate `e1`; only if it fails (`Left`), evaluate `e2`.

Verified results ✓

Expression	Result
`Add (Div 1 0) (Var "x")`	`Left "Division by zero"`
`Add (Var "x") (Div 1 0)`	`Left "Unknown variable: x"`
`ForLoop ("p",e) ("i",0) body`	value of `e`; `body` is never run, so its errors don't matter
`Let "i" (-1) (ForLoop ("p",7) ("i", i+1) (1/0))`	`Right 7`: the bound is `-1+1 = 0` (outer `i`), so the loop runs 0 times
`Let "x" 10 (ForLoop ("x",0) ("i", Var "x") (x+1))`	`Right 10`: the bound uses the outer `x = 10`; inside, `x` is the loop parameter
`ForLoop ("p",0) ("i",10) (p+i)`	`Right 45`
`Apply (Var "nope") (Div 1 0)`	`Left "Unknown variable: nope"`: the function is evaluated first
`Apply (CstInt 1) (Div 1 0)`	`Left "Division by zero"`: the argument error beats "not a function"
`Apply (Lambda "v" 3) (Div 1 0)`	`Left "Division by zero"`: eager argument
`TryCatch (TryCatch (Var "a") (Var "b")) 9`	`Right 9`
`TryCatch (Var "a") (Var "b")`	`Left "Unknown variable: b"`
`let y=1 in let f=\x->x+y in let y=100 in f 1`	`Right 2` (lexical scope)

Dummy exam

- **D 2.1.1:** (1) division-by-zero error; (2) unknown-variable error; (3) the order does not matter whenever **both operands terminate successfully**.

- **D 2.1.2:** (1) it evaluates to `e` no matter what `body` does, since the body runs 0 times. (Only `e` itself failing matters.) (2) **No.** The bound `i + 1` is evaluated with the **outer** `i = -1`, so the bound is 0 and the loop body never runs. A `ForLoop` can never loop forever, because its bound is a fixed integer that is computed once.

Report questions ★

1. **Evaluation order for `Apply`:** function, then argument, then the "is it a `ValFun`" check. A1 has no side effects, so the order is only observable through **which error is reported** when several parts fail, and through **non-termination**: if `e1` fails, a diverging `e2` is never evaluated. Example: `Apply (Var "nope") (1/0)` reports the unknown variable.
2. `TryCatch` **evaluating both subexpressions first?** Not correct in general. If `e1` succeeds, the result must not depend on `e2`, but eagerly evaluating `e2` can **diverge** (and, with effects such as A2's print and store, perform effects that should not happen). It is correct only **if** `e2` **always terminates and has no effects**; then it is just wasted work. (In Haskell, `let r2 = eval env e2` is harmless as long as `r2` is only forced on failure, thanks to laziness.)
3. **Can** `eval` **loop forever?** Yes, through general recursion with functions: `(\x -> x x) (\x -> x x)` (Ω), or a Y-combinator factorial applied to −1. A `ForLoop` alone always terminates.
4. **Equalities:**
   - (a) `Apply (Lambda "v" x) y = x` holds when `v` is **not free in** `x` and `y` **evaluates successfully** (terminates without error). Otherwise the left side either uses `y`'s value or reports `y`'s error.
   - (b) `TryCatch (TryCatch x y) z = TryCatch x y` holds when the inner try **does not fail**, i.e. `x` or `y` succeeds. If both fail, the left side runs `z` and the right side fails with `y`'s error ✓.
5. **Dynamic scope:** in `Apply`, evaluate the body in the **caller's** environment (`envExtend v arg env`) instead of the closure's `f_env`. `ValFun` then no longer needs to store an `Env`.
6. `printExp` **on left-nested** `Add`**: O(n²)**. `++` copies its left argument, and in a left-nested tree the left operand is the long string, so it is copied again at every level. Use `ShowS` / difference lists (`showString`, function composition) or a `Builder` for **O(n)**.
7. **Properties of** `printExp`**:**
   - round trip: `parse (printExp e) == Right e`;
   - weaker: `eval (parse (printExp e)) == eval e`;
   - injective: `e1 ≠ e2 ⇒ printExp e1 ≠ printExp e2`;
   - the output is finite and its parentheses are balanced.
8. **Why** `Val` **is in** `APL.Eval`**, not** `APL.AST`**:** `Val` is a run-time concept (`ValFun` contains an `Env`), not syntax. The parser and printer only need `Exp` and should not depend on the evaluator. The dependency goes Eval → AST, never back, and other interpreters can choose their own value types.

§A2 Monadic evaluator (week 2, compendium p.35–37, A2 text p.40–46)

Facts

```
-- week 2: Reader + Error
newtype EvalM a = EvalM (Env -> Either Error a)
EvalM x >>= f = EvalM $ \env -> case x env of
  Left err -> Left err
  Right x' -> let EvalM y = f x' in y env

-- A2: Reader + State + Error
data State = State { stateOutput :: [String], stateKv :: [(Val, Val)] }
newtype EvalM a = EvalM (Env -> State -> (Either Error a, State))
EvalM m >>= f = EvalM $ \env s -> case m env s of
  (Left err, s') -> (Left err, s')
  (Right x, s')  -> let EvalM m' = f x in m' env s'
askEnv = EvalM $ \env s -> (Right env, s)
localEnv g (EvalM m) = EvalM $ \env s -> m (g env) s
runEval :: EvalM a -> ([String], Either Error a)
```

- `Print s e` prints `"s: v"` (with a space; functions print as `#<fun>`) and returns `v`.
- `KvPut k v` stores the pair, replacing an old one, and returns `v`. `KvGet k` on a missing key → `"Invalid key: ValInt 1"`.
- **The key distinction (D 2.2.1):**
  - `(Either Error a, State)`: the state is returned even on error → **effects survive errors**. Conventional impure languages work like this.
  - `Either Error (a, State)`: no state on error → effects are **rolled back**, like a transaction.
- `mapM` / `sequence` run effects **left to right**, because `>>=` threads the state in that order.
- `checkExp` (A2 Task 3): a Reader of the variables in scope + an error/Maybe monad. `Let` and `Lambda` extend the scope for their body only.

Dummy exam

- **D 2.2.1:** (1) The first keeps state changes when an error occurs; the second discards them. (2) The first resembles conventional impure languages, where state changes are not rolled back.
- **D 2.2.2:** `runEval (eval (Add (Print "a" (CstInt 1)) (Print "b" (CstInt 2))))` = `(["a: 1","b: 2"], Right (ValInt 3))`. The left operand is printed first.
- **D 2.2.3:** with `catch` discarding the state on `Left` (`(Left _, _) -> m2 env state`): `TryCatch (Div (CstInt 1) (Print "a" (CstInt 0))) (Print "b" (CstInt 2))` → `(["b: 2"], Right (ValInt 2))`. The print of `a` happened before the division failed, but `catch` throws that state away. The fix keeps the failing branch's state:

  ```
  catch (EvalM m1) (EvalM m2) = EvalM $ \env state ->
    case m1 env state of
      (Left _, state') -> m2 env state'      -- was: m2 env state
      (Right x, state') -> (Right x, state')
  ```

  Now the result is `(["a: 0","b: 2"], Right (ValInt 2))`.

Report questions ★

1. **Complexity of `Print`:** appending with `output ++ [s]` costs O(length of output), so n prints cost **O(n²)**. Improve by consing (`s : output`) and reversing once in `runEval` (O(n) total), or with a difference list / `Data.Sequence`.

2. **Repeated `KvPut` on the same key:** with "filter out the old pair, then insert", memory stays **constant** (the store size = number of distinct keys). If you just cons a new pair (and `lookup` finds the newest), the store **grows** with every put. (Our group used filter → constant.)
3. **Effects of a failed `e1` visible in `e2`?** It depends only on `catch`. If `catch` runs `m2` with the **original** state, the prints and puts of `e1` are lost (our group's version). Passing `state'` from the failed `m1` makes them visible; see D 2.2.3 for the code. A test: `TryCatch (Let "x" (KvPut 1 100) (Div 1 0)) (KvGet 1)` gives an invalid key if the effects are invisible, and 100 if they are visible.
4. `Monoid (m ())`: yes: `a <> b = a >> b` (i.e. `a >>= \_ -> b`) and `mempty = pure ()`.
   - Associativity follows from monad associativity.
   - `mempty <> a = a` follows from left identity.
   - `a <> mempty = a` follows from right identity, because the result is always `()`, so `\_ -> pure ()` is the same as `pure`.

   Caveats: it needs `FlexibleInstances`, and it **overlaps** existing instances with a different meaning. For example `[()]` already has `Monoid [a]` (append), while list `>>` is a cartesian product.

Variants

- `Let "x" (Print "a" 1) (Var "nope")` → `(["a: 1"], Left "Unknown variable: nope")`: the print survives the error (first `EvalM` form).
- With `Either Error (a, State)` the same program gives no output at all, because the error discards the state.

§A3 Parsing (week 3, A3 text p.74–80, week 7 parser p.220–223)

Facts

- **Lexeme style:** every token parser consumes the whitespace **after** itself (`lexeme p = p <* space`). `parseAPL` does `space *> pExp <* eof`, to skip leading whitespace and to reject trailing garbage.
- `lKeyword s = lexeme $ void $ try $ chunk s <* notFollowedBy (satisfy isAlphaNum)`**.** The keyword must not continue as a name, so `getx` is not `get x`.
- `lVName = lexeme $ try $ do …; if v` **elem** `keywords then fail … else pure v`**.** It reads a name, then **rejects keywords**.
- `try p`**:** if `p` fails *after consuming input*, rewind as if nothing was consumed, so that `choice`/`<|>` can try the next alternative. Without `try`, a failure after consuming input fails the whole `choice`.
- `choice` is **ordered**: the first alternative that succeeds wins. Put an always-succeeding `pure x` last, and `**` before `*`.
- Megaparsec's `chunk`/`string` is atomic: a failed `chunk "**"` consumes nothing.
- **A3 precedence (low → high):** `==` (left), `+ -` (left), `* /` (left), `**` (**right**), application (left, binds tightest). `if`, `\`, `let`, `loop` and `try` extend as far right as possible.

Final grammar ★ Q1 (left recursion removed; `{ }` = repeat, folded left)

```
Atom  ::= var | int | bool | "(" Exp ")"
FExp  ::= Atom { Atom }                          -- application, left-assoc
LExp  ::= "if" Exp "then" Exp "else" Exp
        | "\" var "->" Exp
        | "try" Exp "catch" Exp
        | "let" var "=" Exp "in" Exp
        | "loop" var "=" Exp "for" var "<" Exp "do" Exp
        | "print" string Atom | "get" Atom | "put" Atom Atom
        | FExp
Exp3  ::= LExp [ "**" Exp3 ]                     -- right-assoc: recurse on the right
Exp2  ::= Exp3 { ("*" | "/") Exp3 }
Exp1  ::= Exp2 { ("+" | "-") Exp2 }
Exp0  ::= Exp1 { "==" Exp1 }
Exp   ::= Exp0
```

Left-associative levels are implemented with a `chain` accumulator (G3). Right-associative `**`: `pExp3 = do x <- pLExp; choice [do lString "**"; y <- pExp3; pure (Pow x y), pure x]`. Pitfall: writing `Exp1 ::= Exp2 "+" Exp1` and building the tree directly from it gives **right**-associative `+`. Say that you fold left (accumulator) to get `(a+b)+c`.

Dummy exam

- **D 2.3.1:** (1) `lVName` parses `get` as a name, then rejects it because it is in the keyword list. (2) `lKeyword` requires that the keyword is **not followed by an alphanumeric character**, so `getx` is not the keyword `get`. (3) **Yes**, `if x y then z else w` parses: `lVName` is wrapped in `try`, so trying to read `then` as a variable fails **without consuming input**, the application chain stops, and `then` is then parsed as a keyword.

Report questions ★

2. `*` **is a prefix of** `**`**:** this is only a problem if `*` is tried first. On `x ** y` the parser would read `*` as multiplication and then fail on `* y`. It is fine when `**` is tried first, either because it is a higher precedence level (parsed inside the operands of `*`) or because `**` comes first in an ordered `choice`. Another fix: `lString "*" <* notFollowedBy (chunk "*")`.

3. `try` **necessary vs convenient:**
   - *Necessary:* in `lVName` (D 2.3.1.3, otherwise `if x y then …` fails) and in `lKeyword` (otherwise `iffy` consumes `if`, fails on `notFollowedBy`, and the valid program is rejected).
   - *Convenient:* the `try` between alternatives that share a prefix: `loop … for` vs `loop … while` (week 7), and `(e)` vs `(e1, e2)` (week 7 tuples). These could be removed by **left-factoring**: parse the common prefix once, then `choice` on what follows.
   - *What removing one changes:* without the `try` and without left-factoring, the **language changes** (e.g. while-loops or tuples are no longer accepted). With left-factoring, the language is the same, but the **running time** improves, because `try` re-parses the shared prefix. With nested parentheses that is exponential (`((((1))))` re-parses the inner expression twice per level). **Error messages** also improve, since they report the real position instead of the backtrack point.
4. **Ordered** `choice` **matters:** every `chain` ends with `pure x`, which always succeeds, so it must be **last**. Otherwise no operator is ever consumed and `1+2` fails at `eof`. Likewise `choice [lString "**", lString "*"]` needs `**` first.
5. **Comments:** change only the whitespace parser used by `lexeme` (and by the initial `space` in `parseAPL`), e.g. `L.space space1 (L.skipLineComment "--") (L.skipBlockComment "{-" "-}")`. This is a small change, because every token already goes through `lexeme`. The comment syntax must not clash with an operator; `--` is safe because APL has no unary minus.

Variants

- `x y z` → `Apply (Apply x y) z`. `x (y z)` → `Apply x (Apply y z)`. `x if a then b else c` → parse error, because an `if` is not an `Atom` and needs parentheses as an argument. The same holds for `f \x -> x`.
- `get x + y` → `Add (KvGet x) y`. `x*y**z` → `Mul x (Pow y z)`. `x+y==y+x` → `Eql (Add x y) (Add y x)`. `let true = y in z` → error (keyword).

§A4 Free monads (week 4, A4 text p.109–120, Monad p.218–219 for the week 7 version)

Facts

```
data Free e a = Pure a | Free (e (Free e a))
instance Functor e => Monad (Free e) where
  Pure x >>= f = f x
  Free g >>= f = Free $ fmap (>>= f) g     -- push f into the "rest of program"
```

- **Why `e` must be a `Functor`:** `>>=` must attach the continuation `f` to the rest of the program stored *inside* the effect, and `fmap` is how to reach it.
- **Shape of an effect constructor:**
  - an effect that returns a value carries a continuation `Val -> a` (`KvGetOp Val (Val -> a)`, `ReadOp (Env -> a)`);
  - an effect without a result carries the next program `a` (`KvPutOp Val Val a`, `PrintOp String a`);
  - `ErrorOp Error` has **no** `a`: nothing follows, so `fmap` ignores it and the program stops.
- **A4** `EvalOp` (group hand-in): `ReadOp`, `PrintOp`, `ErrorOp`, `TryCatchOp (EvalM Val) (EvalM Val) (Val -> a)`, `KvGetOp`, `KvPutOp`, `TransactionOp (EvalM Val) (Val -> a)`, `BreakOp Val`.
- **Interpreters:** `runEval'` (pure) passes the key-value `State` as a parameter. `runEvalIO'` keeps it in a **database file**: every `KvPutOp` reads and writes the file, and a missing key **prompts the user** for a value.
- `localEnv f` is not an effect. It walks the effect tree and changes every `ReadOp k` into `ReadOp (k . f)`, recursing into the `EvalM` payloads of `TryCatchOp` and `TransactionOp`.
- `TransactionOp m k`: run `m`; on `Right v` keep its state changes and continue with `k v`; on `Left e`, **roll back** the key-value store but **keep the prints**, and propagate the error. IO version: run `m` on a temporary copy of the DB (`withTempDB` + `copyDB`), and copy it back only on success.
- **Break:** `looping m` delimits a loop and `breakLoop v` returns `v` from the innermost `looping` **at run time** → dynamic scope.

Dummy exam

- **D 2.4.1:**
  1. `ForLoop ("p",0) ("i",10) (Apply (Lambda "x" (Break (Var "i"))) 0)` → `ValInt 0`. The break happens in the first iteration (`i = 0`), even though it is inside a function call.
  2. `Apply (Let "f" (ForLoop ("p",0) ("i",1) (Lambda "x" (Break 7))) (Var "f")) 0` → **error "Break outside loop"**. The loop just returns the lambda, and when it is applied later, no loop is running.
  3. **Dynamic.** "Innermost loop" means the nearest `looping` that is active in the **effect tree at run time**, and that tree is built dynamically as the program runs, not from where the code is written.

Report questions ★

1. `TryCatchOp m1 m2 k`**,** `m1` **fails after key-value effects:**
   - (a) Can differ. The pure interpreter runs `m2` with the state from **before** `m1`, so the effects are invisible. The IO interpreter has already written them to the **database file**, and `m2` reads the same file, so they are visible. (Our group's hand-in: invisible in pure, visible in IO.)
   - (b) Yes, without changing the interpreters: wrap `m1` in a **transaction**, `catch (transaction m1) m2` (APL: `TryCatch (Transaction e1) e2`). The transaction rolls back the store when `m1` fails, then rethrows, and `catch` runs `m2` on the clean store.

2. **Why not `TransactionOp (EvalM ()) a`?** The transaction could not **return a value**: the payload's result is thrown away, and the continuation is a plain `a`, not `Val -> a`. So `Transaction e` could not evaluate to the value of `e`. It is **not useless**: rollback still works for transactions run only for their effects, and a result could be passed out through the key-value store.
3. **Why not `TryCatchOp a a`?** In `Free`, the `a` is the **rest of the program**. `>>=` (via `fmap`) would copy the continuation into **both** branches, so the interpreter cannot see where the try-block ends:
   - errors that happen *after* the try-catch would be caught too;
   - the code after the try-catch could run in both branches, e.g. printing twice.

   Example: `let x = try 0 catch 1 in 10/x` should fail with division by zero but would give 10. With `TryCatchOp (EvalM Val) (EvalM Val) (Val -> a)`, the two blocks are separate values and only `k` is the rest of the program.

Our group's `break` bug (in case it comes up)

`looping` replaced `BreakOp v` with `Pure v` **inside the `TryCatchOp` payload too**. So `ForLoop … (Add (TryCatch (Break 42) 99) 1)` made the try-block succeed with 42, and the loop continued: 43 instead of 42. The fix: a `LoopOp (EvalM Val) (Val -> a)` effect that the interpreter handles like `TryCatchOp`, so that a break propagates like an error until it reaches the loop.

§A5 Property-based testing (week 5, A5 text p.148–152)

Facts (QuickCheck vocabulary)

Function	What it does
`Gen a`, `Arbitrary a` (`arbitrary`, `shrink`)	random generator / default generator + shrinker
`sized :: (Int -> Gen a) -> Gen a`, `resize n g`	read or set the size parameter
`oneof [g]` / `frequency [(w, g)]` / `elements [x]` / `choose (lo,hi)`	uniform choice of generator / weighted choice / pick a value / range
`forAll g p`, `p ==> q`	explicit generator / precondition (discards a test case)
`classify c lbl`, `label`, `collect`, `tabulate`	**only report** the distribution
`cover pct c lbl` + `checkCoverage`	report, **and fail** if fewer than `pct` % hit `c` (statistically)
`within n`, `discardAfter n`	time limit per test (fail / discard)
`withMaxSuccess n`, `counterexample s`	number of tests / extra text when a test fails

- **Recursive generator:** use `sized` and **halve the size** at each recursion level, with a leaf case at size 0. Otherwise terms are huge, or the generator never terminates (if the expected number of children is ≥ 1).
- **Variables from a small pool / from the scope:** random names almost never match a bound name, so every `Var` would be an unknown-variable error. Pass `[VName]` (the names in scope) to `genExp`; `Let` and `Lambda` add to it.
- `shrink`**:** return strictly **smaller** candidates (the subterms `e1` and `e2`, plus the term rebuilt with shrunk children), so that shrinking terminates.
- **Round trip** `parse (printExp e) == Right e` tests the printer **and** the parser at once. A failure is a bug in one of them, or in the generator (it produces something the language does not allow).

Dummy exam

- **D 2.5.1:**
  1. **Yes**, e.g. `(\x -> x x) (\x -> x x)`.
  2. **Unlikely** with a uniform generator, since the space is huge and self-application needs a very specific shape.
  3. Two ways to stop the suite hanging: put a **time limit** on each test (`within`/`discardAfter`/Tasty `mkTimeout`), or **constrain the generator**, e.g. type-directed generation, since self-application is not well-typed.
  4. **With while-loops** an infinite loop becomes **likely**: a tautological condition is enough, and shrinking tends to reduce a condition to `true`. A timeout still works. Otherwise generate only loops with a counter that stops after k iterations.

Report questions ★ (group's answers, condensed)

1. **Can** `genExp` **programs loop?** Yes. A variable bound to `\x -> x x` gives Ω, e.g. `let d = \x -> x x in d d`; about 3 in 10000 programs did. Avoid it with type-directed (well-typed) generation (big change, and it removes the type errors `expCoverage` wants), or make it harmless with `discardAfter`, which is what we did.
2. **Hardest coverage requirement:** "≤ 30% variable errors", because it fights "≥ 50% contain a variable" (one bad `Var` anywhere is an error). Fix: take `Var` from the scope list almost always, and give `Var` a low weight when nothing is in scope. Tune with `classify`/`tabulate` rather than rerunning `checkCoverage`.

3. `parsePrinted` **counterexamples:**
   - negative constants: fix the **generator** (APL has no negative literals);
   - `try` without parentheses inside `/`: fix the **printer**;
   - nested `Apply` printed without parentheses: fix the **printer**;
   - keywords as variable names: fix the **generator**.

   Rule: fix the generator if the input is not a legal program; fix the implementation if a legal program is printed or parsed wrongly. The property was never changed.
4. **Faulty assumption of** `checkExp`**:** it assumes that an error inside `try` is always **caught by that** `try`. A `Lambda` inside `try` succeeds immediately and its body runs **later**, outside the `try`. Counterexample: `Apply (TryCatch (Lambda "nn" (Var "b")) (CstInt 4)) (CstInt 4)`. The checker masks `b`, but evaluation gives `UnknownVariable "b"`.
5. **Class never generated:** expressions where a domain error **actually happens at run time**. The operands of `/` are usually not integers, so evaluation fails with a type error first. Add a run-time `cover` (e.g. `cover 5 (runEval (eval e) == Left DivisionByZero)`), and generate integer-typed operands for `Div`/`Pow`, sometimes `CstInt 0`.

§A6 / W6 Concurrency, GenServer, SPC (compendium p.166–201, notes p.177–217)

Facts

- **Primitives:** `forkIO` (new thread), `Chan` (unbounded FIFO queue; `readChan` blocks when empty), `MVar`, `killThread` (throws the asynchronous exception `ThreadKilled` into the thread).
- **GenServer** (p.181):
  - `spawn :: (Chan a -> IO ()) -> IO (Server a)` starts a server loop in a new thread;
  - `sendTo` is **async**;
  - `requestReply serv (\rc -> Msg … rc)` is **sync**: it makes a reply channel, sends the message, and blocks on the reply;
  - `reply rc x` answers.
- **Server loop pattern:** `loop state = do msg <- receive c; state' <- handle msg state; loop state'`. **All state lives in one thread** and messages are handled one at a time, so each message is handled atomically and there are no data races or locks.
- **SPC (week 6 solution):**
  - the state holds pending jobs, the running job (id, deadline, `ThreadId`), done jobs, and the `jobWait` clients that are waiting;
  - each job runs in a `forkIO`'d thread wrapped in `catch` (`SomeException`) → `MsgJobDone` or `MsgJobCrashed`;
  - `jobAdd`, `jobStatus` and `jobWait` are sync; `jobCancel` is **async**;
  - a **ticker thread** sends `MsgTick` every second. `checkTimeouts` runs before each message, kills a job past its deadline, and marks it `DoneTimeout`. The ticker is needed because a server that only checks when messages arrive would never notice a timeout while no messages come;
  - `jobWait` on an unfinished job stores the reply channel in `spcWaiting` and replies when the job is done;
  - a `MsgJobDone` / `MsgJobCrashed` for a job that is **no longer running** is ignored. This matters after a cancel or timeout: `killThread` makes the job's `catch` handler send `MsgJobCrashed`, which arrives late;
  - in the week 6 solution, `jobCancel` only affects the **running** job. Cancelling a *pending* job is ignored (`_ -> pure ()`), and the job runs later.
- **A6 workers:** each worker is a server thread. SPC keeps the idle and busy workers and a queue of pending jobs. Whenever a job is pending **and** a worker is idle, SPC sends the job to that worker and marks it busy. The worker runs the job (catching exceptions) and reports back with its name.

Dummy exam

- **D 2.6.1 Central vs local timeouts:**
  - *Central* (SPC checks deadlines on each message/tick): no extra thread per job, simpler, one place for the logic. But timeouts are only noticed when a tick or message arrives, so detection is **delayed** (up to the tick interval).
  - *Local* (the worker times out its own job): precise, independent of SPC load. But it needs an extra thread or timer per job plus abort logic in the worker, and a stuck worker may not notice.
  - *Observable:* **how long after the deadline** the status becomes `JobDone DoneTimeout`.

Report questions ★ (generic answers, since no A6 hand-in is in this repo)

1. **Job added with no workers, worker added later:** the job stays `JobPending` in the queue. When `workerAdd` makes an idle worker, SPC immediately schedules the pending job to it (status `JobRunning`), and `jobWait` callers are answered when it finishes. Test: add a job, check `JobPending`, add a worker, `jobWait` → `Just Done`.
2. **Central vs decentral timeouts:** see D 2.6.1. Pros and cons as above; the observable effect is the delay before `DoneTimeout`.
3. **Worker survives a cancelled, timed-out or crashed job:** test with **exactly one** worker. Run a job that is cancelled (or times out, or calls `error`), then run a second job and check that it ends with `Done`. With one worker, the second job can only succeed if the worker survived.
4. **A worker thread itself dies:** SPC is not told, so it still thinks the worker is idle or busy. Its current job never completes (`jobWait` blocks forever, status stays `JobRunning` unless a central timeout eventually marks it `DoneTimeout`). Jobs sent to it later are lost, because a `Chan` write never fails. The rest of SPC keeps working with the other workers, with less capacity. Fix: monitor workers, e.g. `forkIO (worker` finally `send spc (MsgWorkerDied name))`.
5. `jobCancel` **(async) then** `jobStatus`**:** both messages go into the **same FIFO channel** from the same thread, so SPC handles the cancel first; there is no race on message order. The status is `DoneCancelled` **if SPC marks the job cancelled as soon as it handles the cancel**. If SPC waits for the worker to confirm, `jobStatus` can still say `JobRunning`, which is a race. If the job had already finished, it stays `Done`. (In the week 6 code, a cancel of a *pending* job is ignored, so the status would stay `JobPending`.)

Other likely questions

- **Why catch exceptions in the job's own thread?** An exception only kills the thread it is raised in; the server thread is never told. Catching it in the job thread turns it into a `MsgJobCrashed` message.
- **Danger of catching** `SomeException`**:** it also catches `ThreadKilled` from `killThread` (cancel/timeout), so the handler reports "crashed" for a cancelled job. SPC must ignore that late message (`jobDone` checks whether the job is already done or no longer running).
- **If the SPC server thread crashes:** every client blocked in `requestReply` waits forever, and `sendTo` calls are silently lost. If the runtime can see that no live thread can ever reply, it throws *"thread blocked indefinitely in an MVar operation"* in the client instead. The ticker thread keeps the server's channel alive, so often it cannot see that.

§W7 Week 7: tuples, while, &&/||, three interpreters (p.202–239)

Facts

- **Precedence (high → low):** projection `.i` > application > `* /` > `+ -` > `==` > `&&` > `||`. All are left-associative. Parser levels: `pExp4` (`* /`), `pExp3` (`+ -`), `pExp2` (`==`), `pExp1` (`&&`), `pExp0` (`||`), each with a `chain` accumulator.
- **Tuples:** `()` and `(e1, e2, …)` are tuples; `(e)` is just parentheses (there is no 1-tuple in the syntax). The parser tries the tuple alternatives with `try` before `"(" Exp ")"` (p.221). Projection `x.i` is a postfix chain on atoms (`pAtom1`). An index out of range → `"project: invalid index"`; projecting from a non-tuple → `"project: non-pair"`.
- **While:** `loop p = init while cond do body`. `p` is in scope in `cond`; a non-boolean condition is an error.
- `evalStep` is emitted **before every loop body** (for and while) **and before every function-body application**. Application needs it too because recursion through lambdas, like Ω, can loop forever without any loop; the simulator only regains control at a `StepOp`.
- **Effects** (Monad p.218–219): `StepOp a`, `BothOfOp (EvalM Val) (EvalM Val) (Val -> a)` for `&&`, `OneOfOp …` for `||`.

The three interpreters

	Pure (p.215)	Simulated (p.216)	Concurrent (p.213)
`e1 && e2`	`e1` then `e2`; first error	step both in rounds; **fails as soon as either side is an error**	two SPC jobs, waits for **both** (x first)
`e1 || e2`	`e1`; if it fails, `e2`	the side that reaches `Pure` in **fewest steps**; tie → **left**	two SPC jobs, `jobWaitAny`; the winner's value; the **loser is cancelled**
`||` when one side fails	uses the other	fails only if **both** are errors	waits for the other job
`get` on a missing key	**error** `Invalid key`	**blocks** until another branch `put`s it	**blocks** in KVDB until a `kvPut`
`StepOp`	ignored	`step` stops right after it (a "time slice")	ignored
Deterministic?	yes	**yes**	**no**

`step` **in the simulator:** run the computation until just after the next `StepOp`, or until it is `Pure`, an error, or blocked on a `get`, **but no further**. Then nested computations can be **interleaved** round by round. If `step` ran to completion, `(loop … while true …) || 1` would never return.

Verified results ✓ (pure / sim; concurrent reasoned from the code)

Program	Pure	Sim	Concurrent
`(1+2) || (3+4+5+6)`	3	**3**: arithmetic has no steps, so both are `Pure` at once and the tie goes left	3 or 18 (race)
`(loop p = 0 while true do p) || 1`	diverges	**1**	1 (the loop job is cancelled)
`(\x -> x x)(\x -> x x) || 3`	diverges	**3** (thanks to the step at application)	3
`(loop p=0 for i<3 do p+1) || (loop p=0 for i<2 do p+10)`	3	**20** (2 steps beat 3)	3 or 20
`(1+2) && (3+4)`	(3, 7)	(3, 7)	(3, 7)

Program	Pure	Sim	Concurrent
`(1/0) || 5`	5	5	5
`(1/0) && (2/0)`	Div by zero	Div by zero	Div by zero
`(1/0) && (loop … while true …)`	Div by zero (right side never run)	Div by zero	**hangs** (waits for both jobs)
`(1/0) || (loop … while true …)`	diverges	diverges	diverges
`get 0 && put 0 true`	`Invalid key: ValInt 0`	(true, true)	(true, true)
`get 0 + 1 && put 0 2`	Invalid key	(3, 2)	(3, 2)
`put (get 0) 1 && let x = put 0 2 in get 2`	Invalid key	(1, 1)	(1, 1)
`get 0 && get 1`	Invalid key	**stuck forever** (no deadlock detection)	crash: *thread blocked indefinitely in an MVar operation*
`get 0 || 7`	7	7	7
`let _ = (put 0 1 || put 0 2) in get 0`	1	**2**: both sides ran in the same round, and the loser's `put` is **not** undone	1 or 2
`let x = (1,2) in x.0` · `(1,2).1` · `(1,2).2`	1 · 2 · invalid index	same	same

Dummy exam

- **D 2.7.1:**
  1. `(1+2) || (3+4+5+6)` → `ValInt 3`. It is not because the left side has fewer arithmetic operations: **steps are only counted for loops and applications**, and arithmetic has none. Both sides are finished immediately, and the simulator checks the left one first. The simulator is deterministic, so the result is **always** 3.
  2. `(loop p = 0 while true do p) || 1` → `ValInt 1`. The left side never terminates, and the right side finishes after 0 steps. It is **always** 1 (deterministic).

Other likely questions

- **Why `IORef`s in the concurrent interpreter?** An SPC `Job` is an `IO ()` with no result. The job writes its `Either Error Val` into an `IORef`, and the interpreter reads it after `jobWaitAny` says the job is done.
- **Why SPC and not `forkIO`?** It is a requirement, and SPC already provides waiting (`jobWaitAny`), cancellation (killing the loser of `||`) and crash handling. With raw `forkIO` you would rebuild these with `MVar`s.
- **The loser of `||`:** `jobCancel` kills it. If it kept running, it would waste CPU, might never stop (an infinite loop), and could still `put` into KVDB after the result was decided. Its effects from *before* the cancel remain.
- **KVDB** (p.226): a GenServer whose state is the `(k, v)` list plus the **waiters** `[(k, ReplyChan v)]`.
  - `kvGet` is **sync**: if the key is missing, the reply channel is stored and the client blocks.
  - `kvPut` is **async**: it stores the value, replies to all waiters for that key, and removes them.
- **"Thread blocked indefinitely in an MVar operation"** with a correct interpreter: the APL program waits for a key that is never put, e.g. `get 0` alone, or `get 0 && get 1`. Every thread is blocked, so the runtime detects the deadlock. This is not an interpreter bug.
- **Detecting "all blocked" in the simulator:** the solution just loops forever. You could return a flag from `step` saying whether any progress was made, and fail with a "deadlock" error after a round with no progress.
- **Testing:** pure and simulated results can be tested with `@?=`. Concurrent tests must use programs with only one possible result, or accept every allowed result.

My additions (fill in by hand or here)

TA feedback from Absalon (add each point to the matching section too):

A1:

A2:

A3:

A4:

A5:

A6:

Things I got wrong in the drills/mock, in my own words: